

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
SDD
Versione 1.9



Data: 24/11/2025

Progetto: MediBook	Versione: 1.8
Documento: RAD	Data: 24/11/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Lelio Crisci
--------------------	--------------

Revision History

Data	Versione	Descrizione	Autore
24/11/2025	1.1	Aggiunta punto 2 relativo al “Current software architecture”	Samuele Colucci
24/11/2025	1.2	Aggiunta punto 1 relativo all’ “Introduction”	Lelio Crisci
24/11/2025	1.3	Aggiunta punto 3.3 relativo ad “Hardware Software Mapping”	Samuele Colucci
24/11/2025	1.4	Aggiunta punto 3.5 relativo ad “Access control and security”	Lelio Crisci
24/11/2025	1.5	Aggiunta punto 3.2 relativo a “Subsystem decomposition”	Christian Casale
24/11/2025	1.6	Aggiunta punto 3.7 relativo a “Boundary conditions”	Lelio Crisci
24/11/2025	1.7	Aggiunto punto 3.1 relativo a “Overview”	Christian Casale
24/11/2025	1.8	Aggiunto punto 3.4 relativo a “Persistent data management”	Christian Casale
24/11/2025	1.9	Aggiunto punto 4 relativo a “Subsystem services Glossary”	Lelio Crisci

Contents

1. Introduction	4
1.1 Purpose of the system	4
1.2 Design goals	4
1.3 Definitions, acronyms, and abbreviations	5
1.4 References	5
1.5 Overview	6
2. Current software architecture	7
3. Proposed software architecture	8
3.1 Overview	8
3.2 Subsystem decomposition	9
3.3 Hardware/software mapping	10
3.4 Persistent data management	10
3.5 Access control and security	11
3.6 Global software control	12
3.7 Boundary conditions	13
4. Subsystem services	13
4.1 Gestione Utenti	14
4.2 Gestione Prenotazioni	14
4.3 Gestione Referti	15
4.4 Gestione Medico	15
4.5 Gestione Email	15
4.6 Gestione Password	16

System Design

1. Introduction

1.1 Purpose of the system

Il progetto MediBook ha come obiettivo primario la realizzazione di un **sistema digitale di prenotazione delle visite mediche presso uno studio medico**. L'iniziativa nasce per risolvere le criticità legate alla gestione tradizionale degli appuntamenti, che comporta lunghi tempi di attesa per i pazienti, il sovraccarico del centralino e difficoltà di coordinamento tra segreteria e personale medico.

MediBook è progettato per introdurre una **piattaforma digitale centralizzata** che automatizzi la gestione degli appuntamenti, fornisca informazioni aggiornate sulla disponibilità dei medici e riduca il carico di lavoro amministrativo. Il progetto è finalizzato a digitalizzare e ottimizzare l'intero processo di prenotazione, migliorando la qualità complessiva del servizio e la soddisfazione dei pazienti.

1.2 Design goals

Il sistema è progettato facendo riferimento ai seguenti obiettivi, derivati dai requisiti non funzionali e dai vincoli di progetto:

Obiettivi di performance

- **OP1.0 (Reattività e Scalabilità)**: Il sistema deve garantire tempi di risposta stabili e supportare fino a 200 utenti concorrenti senza degrado significativo delle prestazioni, mantenendo la fluidità operativa per le funzioni critiche.
- **OP2.0 (Portabilità)**: Il sistema deve essere accessibile e pienamente funzionale sui principali browser web moderni (Chrome, Edge, Firefox, Safari) in ambiente desktop, senza richiedere installazioni aggiuntive lato client.

Obiettivi di affidabilità

- **OA1.0 (Sicurezza e Compliance)**: Il sistema deve garantire la riservatezza dei dati personali e sanitari in conformità al GDPR, assicurando che le credenziali utente siano memorizzate esclusivamente in formato crittografato nel database.
- **OA2.0 (Robustezza e Integrità)**: Il sistema deve prevedere inconsistenze nei dati gestendo correttamente gli accessi concorrenti, impedendo in particolare la

duplicazione delle prenotazioni sullo stesso slot orario.

Obiettivi di manutenzione

- **OM1.0 (Modificabilità):** Il sistema deve essere sviluppato con un'architettura modulare e corredato da documentazione tecnica (Javadoc), permettendo l'individuazione e la correzione di errori o l'implementazione di modifiche semplici entro tempi ridotti.
- **OM2.0 (Standardizzazione):** Il sistema deve basarsi su tecnologie standard e diffuse (Java, Spring Boot, MySQL) per facilitare il reperimento di risorse tecniche e garantire la longevità del software.

Obiettivi utente finale

- **OU1.0 (Usabilità):** Il sistema deve presentare un'interfaccia intuitiva che permetta agli utenti, anche con competenze informatiche di base, di completare le operazioni principali (es. prenotazione) con un numero ridotto di interazioni (massimo 3 click).
- **OU2.0 (Apprendimento):** Il sistema deve essere progettato in modo tale da ridurre al minimo la curva di apprendimento, permettendo al personale di segreteria di operare autonomamente dopo una breve formazione.

1.3 Definitions, acronyms, and abbreviations

- SSD: System Design Document.
- GDPR: General Data Protection Regulation.
- DG: Design Goal.
- RAD: Requirements Analysis Document.
- RNF: Requisiti Non Funzionali.

1.4 References

- Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd edition.
- Slide del corso di Ingegneria del Software (Prof. De Lucia).
- Documento RAD.
- Requisiti non funzionali: Sezione 3.3 del RAD

1.5 Overview

- Il primo capitolo contiene l'obiettivo del sistema, la definizione dei design goal e una sezione dedicata agli acronimi per una maggiore chiarezza nella lettura della documentazione.
- Il secondo capitolo contiene una panoramica sull'architettura dei sistemi simili.
- Il terzo capitolo contiene la panoramica sull'architettura del sistema proposto, la decomposizione in sottosistemi, la mappatura hardware/software, la gestione dei dati persistenti, il controllo degli accessi, il controllo del flusso globale del sistema e le condizioni limite
- Il quarto capitolo contiene i servizi offerti dai sottosistemi.

2. Current software architecture

Attualmente, la gestione delle prenotazioni e delle prestazioni ambulatoriali presso la struttura ospedaliera non è supportata da un sistema informativo integrato specificamente progettato. I processi sono svolti prevalentemente in modalità manuale e cartacea.

Il flusso di lavoro corrente prevede le seguenti modalità operative:

- **Prenotazione:** Il paziente deve recarsi fisicamente presso lo sportello della segreteria o tentare il contatto telefonico.
- **Gestione Agenda:** L'operatore di segreteria verifica la disponibilità consultando agende cartacee o file non strutturati, inserisce manualmente i dati del paziente e assegna lo slot temporale.
- **Erogazione e Refertazione:** Il medico riceve giornalmente una lista cartacea dei pazienti prenotati. Al termine della visita, il referto viene spesso redatto su supporto cartaceo o su file di testo non collegati all'anagrafe centrale, richiedendo al paziente di tornare in struttura per il ritiro fisico del documento firmato.

3. Proposed software architecture

3.1 Overview

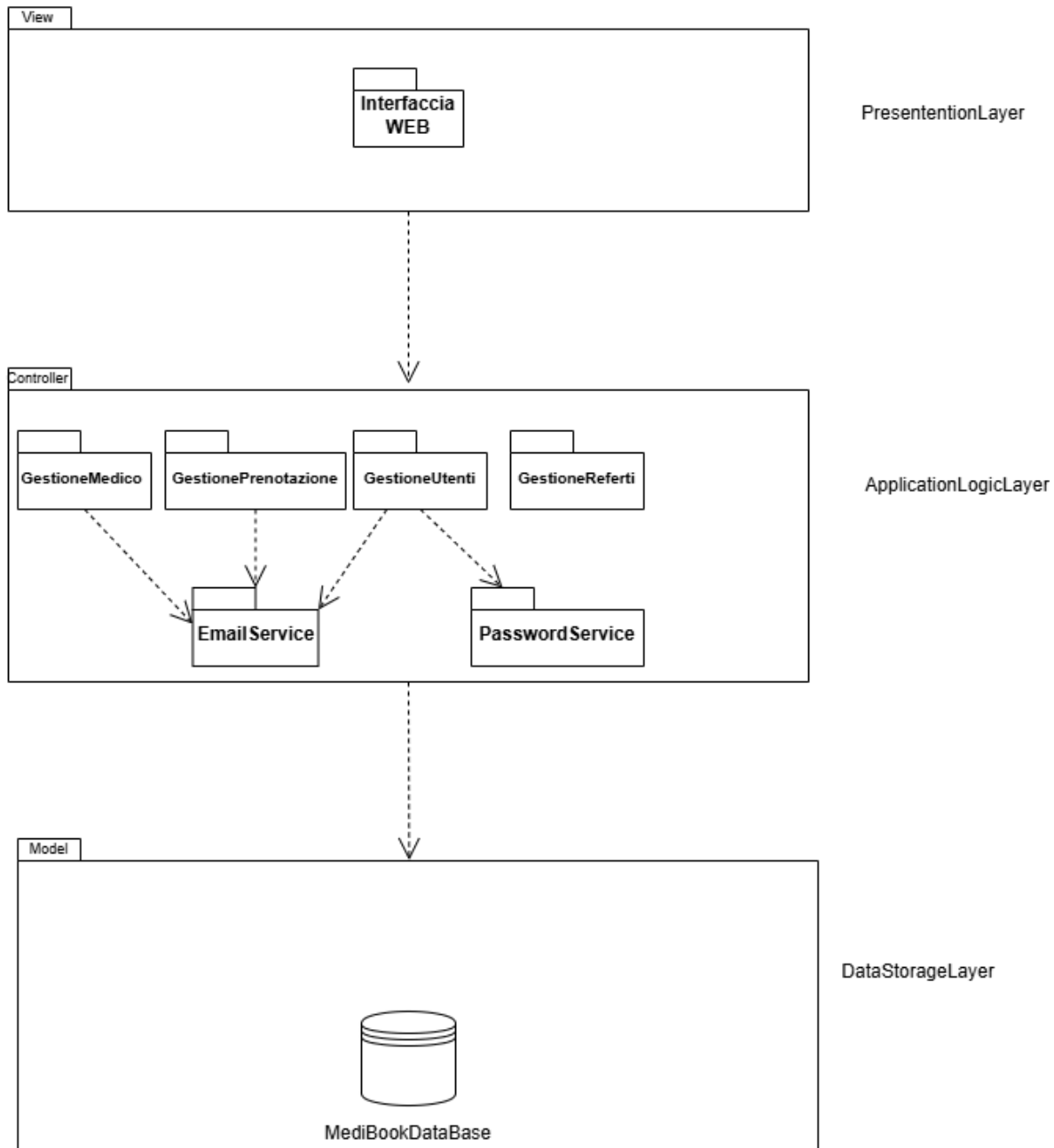
Il sistema **MediBook** è progettato come una piattaforma digitale centralizzata web-based, il cui obiettivo primario consiste nel facilitare i processi di gestione dello studio medico. Il sistema sostituisce i flussi di lavoro manuali con una soluzione automatizzata per ridurre i tempi di attesa, eliminare gli errori di coordinamento e ottimizzare il carico di lavoro amministrativo.

Per garantire modularità, scalabilità e facilità di manutenzione, l'architettura logica del sistema adotta il pattern architetturale **MVC (Model-View-Controller)**:

- **Model (Dati e Logica di Business):** Gestisce le entità fondamentali del dominio, quali l'**Agenda Digitale** (per la sincronizzazione in tempo reale degli appuntamenti e la prevenzione di doppie prenotazioni), le anagrafiche pazienti e i **Referti Digitali**.
- **View (Interfaccia Utente):** Fornisce interfacce web responsive differenziate per attore. Ad esempio, il portale pazienti per le **Prenotazioni Online** e la dashboard amministrativa per la segreteria e i medici.
- **Controller (Gestione Flusso):** Gestisce le interazioni dell'utente, come la richiesta di un appuntamento o il salvataggio di un referto, coordinando l'aggiornamento del Model e selezionando la View appropriata.

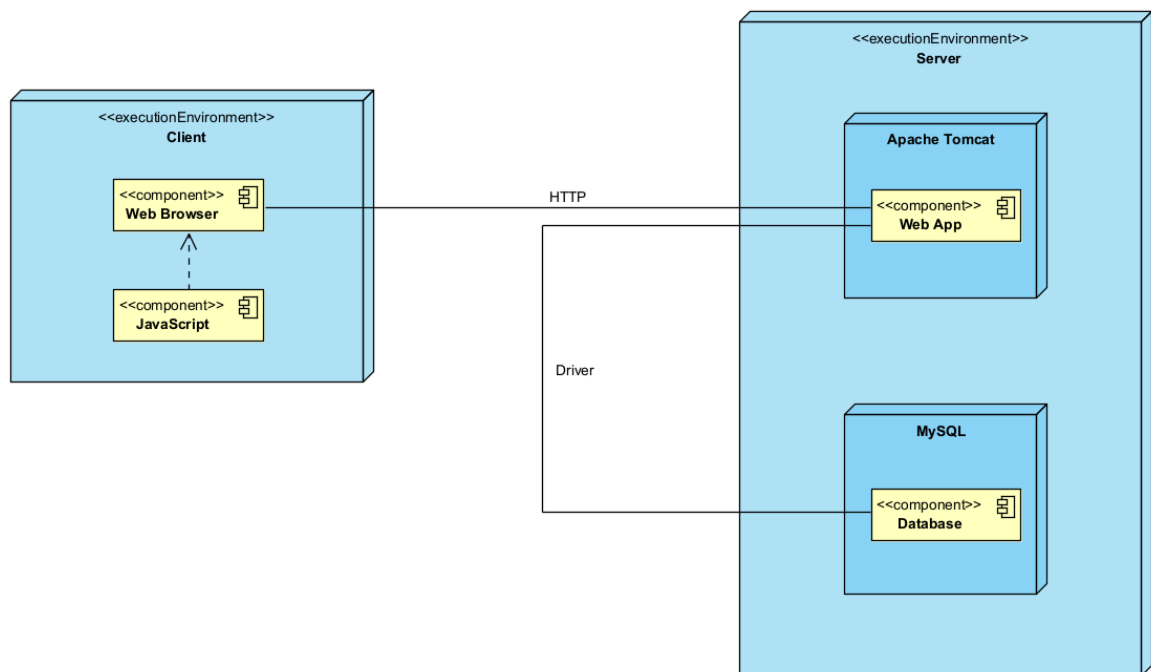
3.2 Subsystem decomposition

Il sistema è stato suddiviso nei seguenti sottosistemi:



3.3 Hardware/software mapping

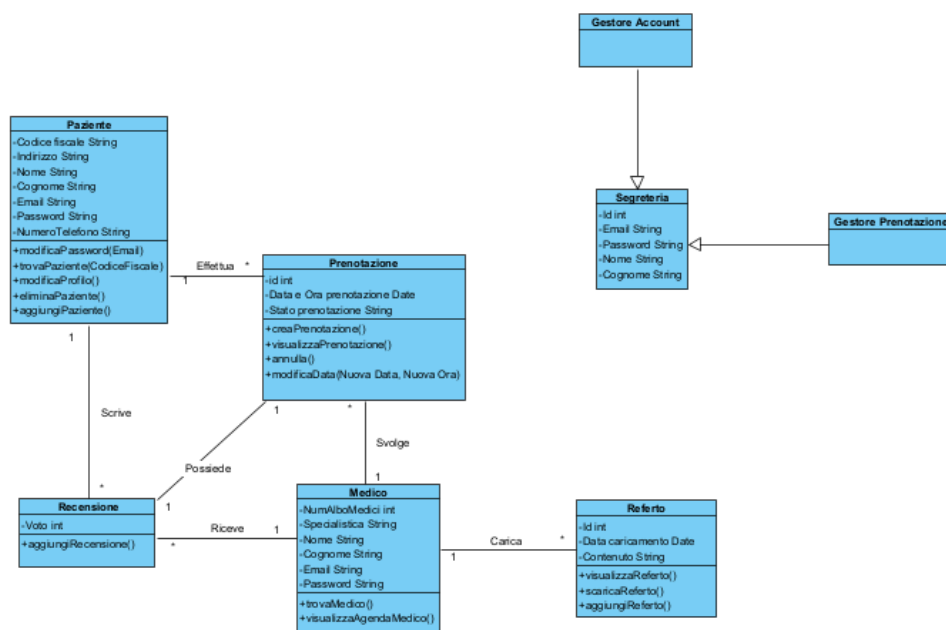
Il sistema MediBook è implementato su un'architettura Client/Server a tre livelli (3-Tier) che separa l'interfaccia utente, la logica applicativa e la gestione dei dati. Il nodo Client è qualsiasi dispositivo che utilizza un browser web compatibile per inviare richieste al Server tramite HTTP. Il nodo Application Server (ambiente Linux con Java 17+ e Apache Tomcat) esegue tutta la logica di business. Su questo stesso nodo, o su un nodo dedicato logicamente separato, risiede il database relazionale MySQL 8.0, che gestisce lo storage dei dati persistenti (incluse agende e referti). L'Application Server interagisce con MySQL tramite driver specifici (come JDBC per Java) per garantire l'accesso rapido e transazionale ai dati, minimizzando i ritardi di rete grazie al posizionamento strategico dei componenti del Backend e assicurando che siano rispettati i requisiti di sicurezza (GDPR) e di scalabilità per gli utenti concorrenti.



3.4 Persistent data management

Per la gestione dei dati persistenti del sistema MediBook, si è scelto di utilizzare un Database Relazionale basato sui principi dell'algebra relazionale, dove le informazioni sono organizzate in tabelle bidimensionali. Il sistema utilizzerà MySQL 8.0 come DBMS, come definito nei requisiti di implementazione.

Come risultato della fase di analisi, di seguito vengono riportati gli oggetti che dovranno essere memorizzati in maniera persistente:



3.5 Access control and security

MediBook garantisce il controllo degli accessi mediante un meccanismo di autenticazione basato su email e password. Solo gli utenti registrati e autenticati possono usufruire dei servizi specifici associati al loro ruolo (Paziente, Medico, Segreteria) e accedere alle aree riservate per la gestione delle prenotazioni e dei dati clinici.

Di seguito viene proposta una tabella dove sono elencate le operazioni che gli utenti possono effettuare in base al loro livello di accesso:

	Visitatore	Paziente	Medico	Segreteria Prenotazioni	Segreteri a Utenti
Utente	Registrazione	Accesso, Modifica Dati	Accesso	Accesso	Creazione Account
Prenotazione	-	Creazione	Visualizzazione, Esecuzione, Annullamento	Visualizzazione, Modifica	-
Referto	-	Visualizzazione, Stampa	Scrittura, Visualizzazione	-	-
Recensione	-	Votazione, Scrittura	Visualizzazione	-	-

3.6 Global software control

Il controllo del flusso software globale di MediBook è gestito dal web server Apache Tomcat. Per ogni utente che si connette e invia una richiesta Tomcat crea un thread dedicato. Questo thread è responsabile di gestire la richiesta specifica, di coordinare le chiamate procedurali ai vari sottosistemi e di rispondere agli eventi generati. È fondamentale, tuttavia, tenere conto delle sezioni critiche che si verificano quando più thread tentano di modificare dati condivisi. Per garantire la consistenza dei dati, il sistema impiega la gestione transazionale e meccanismi di locking a livello del database, assicurando che le operazioni siano atomiche e isolate.

3.7 Boundary conditions

Questa sezione descrive il comportamento del sistema MediBook nelle fasi di transizione (avvio e arresto) e le strategie di resilienza adottate in caso di malfunzionamenti, anomalie o condizioni di carico eccezionali.

Inizializzazione (Startup)

All'avvio del container Apache Tomcat, l'applicazione Spring Boot esegue in sequenza:

- Caricamento Configurazione: Lettura dei parametri sicuri da application.properties.
- Verifica Risorse: Inizializzazione del pool di connessioni al database e controllo permessi di scrittura sullo storage.
- Avvio Servizio: Solo se i check precedenti hanno successo, il server apre le porte HTTP agli utenti; in caso contrario, l'avvio si interrompe.

Chiusura Controllata

La terminazione del sistema deve avvenire in modalità controllata per prevenire la corruzione dei dati. Alla ricezione di un segnale di arresto, il sistema esegue:

- Stop del Traffico in Ingresso: Il server smette di accettare nuove richieste HTTP, restituendo un codice di stato appropriato per le nuove connessioni.
- Draining delle Transazioni: Il sistema attende un tempo prefissato affinché i thread attivi completino le operazioni in corso. Se l'operazione non termina entro il timeout, la transazione viene forzata al rollback per mantenere l'atomicità.
- Rilascio Risorse: Vengono chiusi ordinatamente il pool di connessioni al database MySQL, gli handle di file aperti e i thread di background, garantendo che nessun dato rimanga nel buffer di scrittura volatile.

Gestione delle Anomalie

- Crash e Recovery: In caso di arresto anomalo, la consistenza è garantita dalle transazioni ACID del database.
- Gestione Carico: Il container Tomcat è configurato con un pool di thread limitato (es. server.tomcat.threads.max) per gestire fino a 200 utenti concorrenti. Richieste eccedenti vengono accodate fino al riempimento della coda, oltre il quale il server risponde con HTTP 503 (Service Unavailable).

4. Subsystem services

Questa sezione descrive i servizi forniti dai sottosistemi identificati nella decomposizione del sistema. Ogni sottosistema espone un'interfaccia pubblica che permette agli altri componenti di accedere alle funzionalità principali di MediBook.

4.1 Gestione Utenti

Servizio	Descrizione
login(email, password)	Permette all'utente di autenticarsi tramite email e password, verificando le credenziali.
logout()	Invalida la sessione dell'utente, impedendo ulteriori accessi fino a una nuova autenticazione.
registrazione(datiPaziente)	Permette di creare un nuovo account, creando un nuovo record utente nel database, con email e codice fiscale univoci.
getDatiPaziente(codiceFiscale)	Recupera le informazioni dell'utente.
modificaProfilo(datiInseriti)	Permette di modificare i dati di un paziente.
aggiornaPassword(nuovaPassword)	Permette di aggiornare la password.
trovaByEmail(email)	Permette di trovare gli utenti dalle mail.

4.2 Gestione Prenotazioni

Servizio	Descrizione
findMedico(idMedico)	Permette di ricercare i medici.
nuovaPrenotazione(idPaz,idMed,data,ora)	Permette la registrazione di una nuova prenotazione.
ModificaPrenotazione(idPre,newData,newOra,newStato)	Permette di modificare una prenotazione non ancora svolta.
AggiornaStatoPrenotazione(idPre,newStato)	Il medico e la segreteria hanno la possibilità di aggiornare lo stato di una visita.

4.3 Gestione Referti

Servizio	Descrizione
caricaReferto(refer, prenotazione)	Permette di aggiungere un referto dopo aver eseguito una visita, aggiornando lo stato di una visita contrassegnandola come "Refertata".
visualizzaReferto(refertoId)	Permette di visualizzare il referto una volta inserito.

4.4 Gestione Medico

Servizio	Descrizione
getListaMedici()	Permette di restituire la lista contenente tutti i medici nel sistema.
getMedico(numAlboMedici)	Permette di restituire il medico in base all'albo dei medici.

4.5 Gestione Email

Servizio	Descrizione
inviaEmailModificaPrenotazione(destinatario, oggetto, nomePaziente, nomeMedico, nuovaData, nuovaOra)	Permette di inviare una mail per avvisare del cambio di data di una prenotazione al paziente interessato.
inviaEmailConferma(destinatario, nomePaziente, nomeMedico, data, ora)	Permette di inviare una mail di conferma prenotazione.
inviaEmailModificaMedico(destinatarioMedico, nomePaziente, nuovaData, String nuovaOra)	Permette di inviare una mail per avvisare del cambio di data di una prenotazione al medico interessato.

4.6 Gestione Password

Servizio	Descrizione
hashPassword>Password)	Permette di rispettare la Conformità GDPR al fine di avere la password crittografata.
check>PasswordInserita>PasswordDB)	Permette di controllare se la password inserita sia corretta, controllando nel database tale corrispondenza.